

Integration of Neural Network-Based Symbolic Regression in Deep Learning for Scientific Discovery

Samuel Kim^{1,a}, Peter Lu², Srijan Mukherjee², Michael Gilbert¹, Li Jing², Vladimir Čeperić³, Marin Soljačić²

Abstract—Symbolic regression is a powerful technique that can discover analytical equations that describe data, which can lead to explainable models and generalizability outside of the training data set. In contrast, neural networks have achieved amazing levels of accuracy on image recognition and natural language processing tasks, but are often seen as black-box models that are difficult to interpret and typically extrapolate poorly. Here we use a neural network-based architecture for symbolic regression that we call the Sequential Equation Learner (SEQL) network and integrate it with other deep learning architectures such that the whole system can be trained end-to-end through backpropagation. To demonstrate the power of such systems, we study their performance on several substantially different tasks. First, we show that the neural network can perform symbolic regression and learn the form of several functions. Next, we present an MNIST arithmetic task where a separate part of the neural network extracts the digits. Finally, we demonstrate prediction of dynamical systems where an unknown parameter is extracted through an encoder. We find that the EQL-based architecture can extrapolate quite well outside of the training data set compared to a standard neural network-based architecture, paving the way for deep learning to be applied in scientific exploration and discovery.

Index Terms—Symbolic Regression, Neural Network, Kinematics, Simple Harmonic Oscillator, ODE, Discovery

I. INTRODUCTION

Many complex phenomena in science and engineering can be reduced to general models that can be described in terms of relatively simple mathematical equations. For example, classical electrodynamics can be described by Maxwell's equations and non-relativistic quantum mechanics can be described by the Schrödinger equation. These models elucidate the underlying dynamics of a particular system and can provide general predictions over a very wide range of conditions. On the other hand, modern machine learning techniques have become increasingly powerful for many tasks including image recognition and natural language processing, but the neural network-based architectures in these state-of-the-art techniques are black-box models that often make them difficult for use in scientific exploration. In order for machine learning to be widely applied to science, there is a need for interpretable and generalizable models that can extract meaningful information

from complex datasets and extrapolate outside of the training dataset.

Symbolic regression is a type of regression analysis that searches the space of mathematical expressions to find the best model that fits the data. It is much more general than linear regression in that it can potentially fit a much wider range of datasets and does not rely on a set of predefined features. Assuming that the resulting mathematical expression correctly describes the underlying model for the data, it is easier to interpret and can extrapolate better than black-box models such as neural networks. Symbolic regression is typically carried out using techniques such as genetic programming, in which a tree data structure representing a mathematical expression is optimized using evolutionary algorithms to best fit the data [1]. Typically in the search for the underlying structure, model accuracy is balanced with model complexity to ensure that the result is interpretable and does not overfit the data. This approach has been used to extract the underlying laws of physical systems from experimental data [2]. However, due to the combinatorial nature of the problem, genetic programming does not scale well to large systems and can be prone to overfitting.

Alternative approaches to finding the underlying laws of data have been explored. For example, sparsity has been combined with regression techniques and numerically-evaluated derivatives to find partial differential equations (PDEs) that describe dynamical systems [3]–[5].

There has also been significant work on designing neural network architectures that are either more interpretable or more applicable to scientific exploration. Neural networks with unique activation functions that correspond to functions common in science and engineering have been used for finding mathematical expressions that describe datasets [6], [7]. A deep learning architecture called the PDE-Net has been proposed to predict the dynamics of spatiotemporal systems and produce interpretable differential operators through constrained convolutional filters [8], [9]. [10] propose a neural network module called the Neural Arithmetic Logic Unit (NALU) that introduces inductive biases in deep learning towards arithmetic operations so that the architecture can extrapolate well on certain tasks. Neural network-based architectures have also been used to extract relevant and interpretable parameters from dynamical systems and use these parameters to predict the propagation of a similar system [11], [12].

Here we present a neural network architecture for symbolic regression that is integrated with other architectures so that it

¹Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology, Cambridge, MA, USA

²Department of Physics, Massachusetts Institute of Technology, Cambridge, MA, USA

³Department of Electrical Engineering and Computing, University of Zagreb, Zagreb, Croatia

^aE-mail: samkim@mit.edu

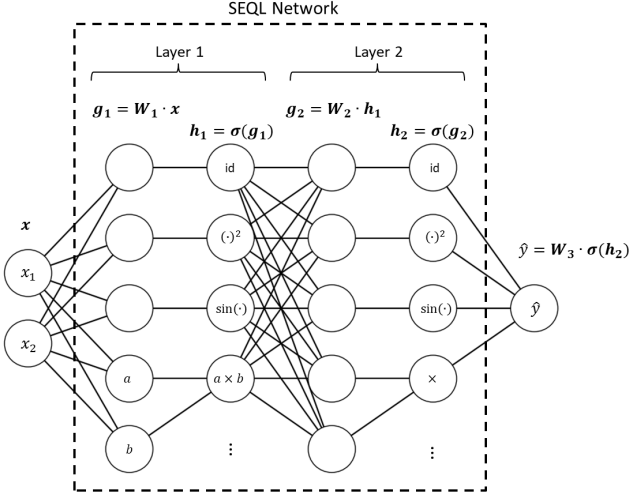


Fig. 1. Example of the Sequential Equation Learner (SEQL) network for symbolic regression using a neural network. Here we show only 4 activation functions (identity or “id”, square, sine, and multiplication) and 2 hidden layers for visual simplicity, but the network can include more functions or more hidden layers to fit a broader class of functions.

can take advantage of the powerful deep learning techniques while still producing interpretable and generalizable results. Sparsity regularization techniques are applied to our symbolic regression neural network to produce simple and interpretable expressions. Because this symbolic regression method can be trained through backpropagation, the entire system can be trained end-to-end.

II. SEQL ARCHITECTURE

The symbolic regression neural network we use is similar to the Equation Learner (EQL) network proposed in [6], [7]. Since our system is designed to be integrated with other deep learning architectures, we will refer to our neural network as the Sequential Equation Learner (SEQL). As shown in Figure 1, the SEQL network is based on a fully-connected neural network where the i^{th} layer of the neural network is described by

$$\begin{aligned} \mathbf{g}_i &= \mathbf{W}_i \mathbf{h}_{i-1} \\ \mathbf{h}_i &= f(\mathbf{g}_i) \end{aligned}$$

where $\mathbf{W}_i$ is the weight matrix of the i^{th} layer and $\mathbf{h}_0 = \mathbf{x}$ is the input data. The final layer does not have an activation function, so for a network with L hidden layers, the output is described by

$$y = \mathbf{h}_{L+1} = \mathbf{W}_{L+1} \mathbf{h}_L$$

The activation function $f(\mathbf{g})$, rather than being the usual choices in neural networks such as ReLU or tanh, may consist of a separate function for each component of $\mathbf{g}$ (such as sine or the square function) and may include functions that take

two or more arguments while producing one output (such as the multiplication function):

$$f(\mathbf{g}) = \begin{bmatrix} f_1(g_1) \\ f_2(g_2) \\ \vdots \\ f_{n_h}(g_{n_h-1}, g_{n_h}) \end{bmatrix}$$

Note that the bias term can be absorbed into $f(\mathbf{g})$ for convenience. These activation functions are analogous to the primitive functions in symbolic regression. Allowing functions to take more than one argument allows for multiplicative operations inside the network.

While the schematic in Figure 1 only shows 4 activation functions in each hidden layer for visual simplicity, the SEQL network can include other functions including $\exp(g)$ and $\text{sigmoid}(g) = \frac{1}{1+e^{-g}}$. This is different from the original EQL network proposed in [6] which consisted of the identity function, sine, cosine, and the multiplication function. Another difference between the SEQL network and the original EQL network is that we allow for activation functions to be duplicated. This reduces the system’s sensitivity to random initializations and creates a smoother optimization landscape so that the network does not get stuck in local minima as easily. These modifications allow the SEQL network to fit a significantly broader range of functions compared to the original EQL network. More details can be found in Appendix A.

By stacking multiple layers, this architecture can fit complex combinations and compositions of a variety of primitive functions. L is analogous to the maximum tree depth in genetic programming approaches and sets the limit on the allowed complexity of the resulting expression. While this model is not as general as conventional symbolic regression, it is powerful enough to represent most of the functions that are typically seen in science and engineering. More importantly, because the SEQL network can be trained by backpropagation, it can be integrated with other neural network-based models for end-to-end training.

A. Sparsity

A key ingredient of making symbolic regression interpretable is enforcing sparsity such that the system finds the simplest possible equation that fits the data. The aim of sparsity is to set as many parameters to 0 such that those parameters are inactive. Enforcing sparsity in neural networks is an active field of research as modern deep learning architectures using millions of parameters start to become computationally prohibitive [13]–[15]. [16] evaluate several recent developments in neural network sparsity techniques.

A straightforward way of enforcing sparsity is adding a regularization term to the loss function where the regularization term is neural network weight values:

$$L_q = \sum_{i=0}^{L+1} \|\mathbf{W}_i\|^q$$

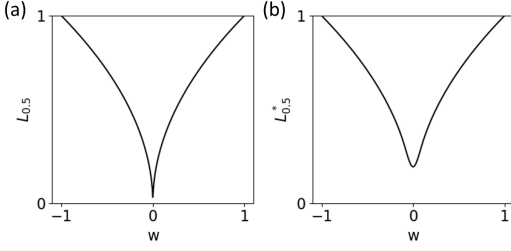


Fig. 2. (a) $L_{0.5}$ and (b) $L_{0.5}^*$ regularization. The threshold is set to $a = 0.1$ for easy visualization.

$\|\mathbf{W}_i\|^q$ is the element-wise norm of the matrix:

$$\|\mathbf{W}_i\|^q = \sum_{j,k} |w_{j,k}|^q$$

L_0 regularization penalizes weights for being non-zero with no further restriction, and thus drives the solution towards sparsity. However, L_0 regularization is equivalent to a combinatorics problem that is NP-hard, and is not compatible with gradient descent methods commonly used for optimizing neural networks [17]. Recent works have explored training sparse neural networks with L_0 regularization through stochastic gate variables [13], [18].

A much more popular and well-known sparsity technique is L_1 regularization, which is used in the original EQL network. Although it is not as sparse as L_0 regularization, L_1 regularization is a convex optimization problem that can be solved using a wide range of optimization techniques including gradient descent to drive the weights towards 0. However, while L_1 is known to push the solution towards sparsity, it has been suggested that $L_{0.5}$ enforces sparsity more strongly without penalizing the magnitude of the weights as much [19], [20]. $L_{0.5}$ regularization is still compatible with gradient descent (although it is no longer convex) and has been applied to neural networks [21], [22]. Experimental studies suggest that $L_{0.5}$ regularization performs no worse than other L_q regularizers for $0 < q < 0.5$, so $L_{0.5}$ is optimal for sparsity [20]. Our experiments with $L_{0.3}$ and $L_{0.7}$ regularizers show no significant overall improvement compared to the $L_{0.5}$ regularizer, in agreement with this study.

In particular, we use a smoothed $L_{0.5}$ proposed in [22], and we label their approach as $L_{0.5}^*$. The original $L_{0.5}$ regularization has a singularity in the gradient as the weights go to 0, which can make training difficult for gradient descent-based methods. To avoid this, the $L_{0.5}^*$ regularizer uses a piecewise function to smooth out the function at small magnitudes:

$$L_{0.5}^*(w) = \begin{cases} |w|^{1/2} & w \geq a \\ \left(-\frac{w^4}{8a^3} + \frac{3w^2}{4a} + \frac{3a}{8}\right)^{1/2} & w < a \end{cases}$$

A plot of the $L_{0.5}$ and $L_{0.5}^*$ regularization are shown in Figure 2. The smoothed regularization avoids the extreme gradient values to improve training convergence. In our experiments, we set $a = 0.01$. When the SEQL network is integrated with other deep learning architectures, the regularization is only applied to the weights of the SEQL network.

We have also implemented an SEQL network with the relaxed L_0 regularization proposed by [13], the details of which can be found in Appendix B.

III. EXPERIMENTS

A. Symbolic Regression

To validate the SEQL network's ability to perform symbolic regression, we first test the SEQL network on data generated by analytical functions such as $\exp(-x^2)$ or $x_1^2 + \sin(2\pi x_2)$. The data is generated on the domain $x_i \in [-1, 1]$. Because of the network's sensitivity to random initialization of the weights, we run 20 trials for each function. We then count the number of times the network has converged to the correct answer ignoring small terms and slight variations in the coefficients from the true value. Additionally, equivalent answers (such as $\sin(4\pi x)$ instead of $\sin(2\pi x)$) are counted as correct. These results are shown in Appendix A.

The network only needs to be able to find the correct answer at least once over a reasonable number of trials, as one can construct a system that picks out the desired equation from the different trials by a combination of equation simplicity and generalization ability. The generalization ability is measured by the equation error evaluated on the domain $x_i \in [-2, 2]$. This extrapolation error of the correct equation tends to be orders of magnitude lower than that of other equations that the network may find, making it simple to pick out the correct answer.

The network is still able to find the correct answer when 10% noise is added to the data. We also test an SEQL network with 3 hidden layers which still finds the correct expression and is able to find even more complicated expressions such as $(x_1 + x_2 x_3)^3$.

B. MNIST Arithmetic

In the first experiment, we demonstrate the ability to combine symbolic regression and image recognition through an arithmetic task on MNIST digits. MNIST, a popular dataset for image recognition, can be notated as $\mathcal{D} = \{\chi, \psi\}$, where χ are 28×28 greyscale images of handwritten digits and $\psi \in \{0, 1, \dots, 9\}$ is the integer-value label. Here, we wish to learn a simple arithmetic function, $y = \psi_1 + \psi_2$, with the corresponding images $\{\chi_1, \chi_2\}$ as inputs and train the system end-to-end.

The architecture is shown in Figure 3. The input to the system consists of two MNIST digits, $x = \{\chi_1, \chi_2\}$. During training, χ_i is randomly drawn from the MNIST training dataset. Each of $\{\chi_1, \chi_2\}$ are fed separately into an encoder to produce single-dimensional latent variables $\{z_1, z_2\}$ that are not constrained and can take on any real value, $z_{1,2} \in \mathbb{R}$. Alternatively, one can think of the system as having a separate encoder for each digit, where the two encoders share the same weights, as illustrated in Figure 3. The encoder consists of two convolutional layers with max pooling layers followed by two fully-connected layers and a batch normalization layer at the output. More details on the encoder can be found in Appendix C. The latent variables $\{z_1, z_2\}$ are then fed as inputs into the

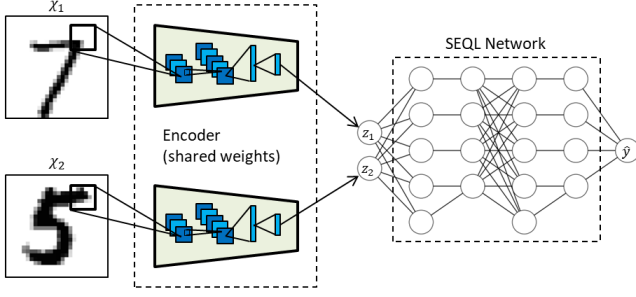


Fig. 3. Schematic of the MNIST addition architecture. An encoder consisting of convolutional layers and fully-connected layers operate on each MNIST image and extract a single-dimensional latent variable. The two encoders share the same weights. The two latent variables are then fed into the SEQL network. The entire system is fed end-to-end and without pre-training.

SEQL network. The SEQL network has a single scalar output $\hat{y}$ which is compared to the true label $y = \psi_1 + \psi_2$.

The entire network is trained end-to-end using a mean-squared error loss between the predicted label $\hat{y}$ and the true label y . Note that the encoder closely resembles a simple convolutional neural network used for classifying MNIST digits except that it outputs a scalar value instead of logits that encode the digit. Additionally, there is no constraint on the properties of $z_{1,2}$, but we expect that it has a one-to-one mapping to the true label $\psi_{1,2}$.

C. Dynamical System Analysis

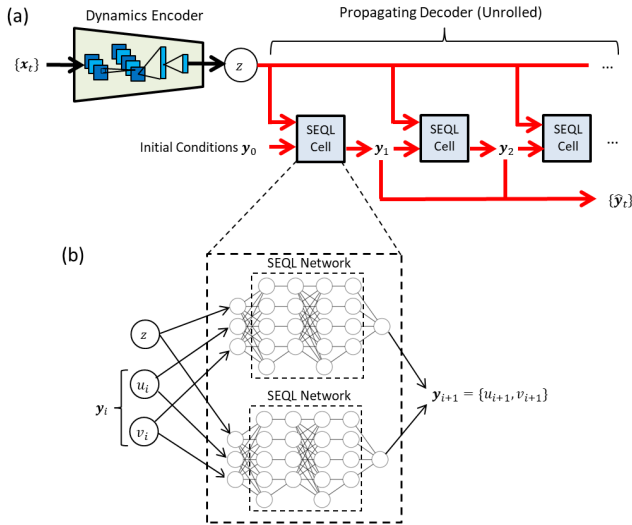


Fig. 4. (a) Architecture to learn the equations that propagate a dynamical system. (b) Each SEQL cell in the propagating decoder consists of a separate SEQL network for each dimension of $\mathbf{y}$ to be predicted. In our case, $\mathbf{y} = \{u, v\}$ where u is the position and v is velocity, so there are 2 SEQL networks in each SEQL cell.

A potentially powerful application of deep learning in science exploration and discovery is discovering parameters in a dynamical system in an unsupervised way and using these parameters to predict the propagation of a new system. For example, [11] use multilayer perceptrons to extract relevant

properties from a system of bouncing balls (such as the mass of the balls or the spring constant of a force between the balls) and simultaneously predict the trajectory of a different set of objects. [12] accomplish a similar goal but using a dynamics encoder (DE) with convolutional layers and a propagating decoder (PD) with deconvolutional layers enable analysis and prediction of spatiotemporal systems such as those governed by PDEs. This architecture is designed to analyze spatiotemporal systems that may have an uncontrolled dynamical parameter that varies among different instances of the dataset such as the diffusion constant in the diffusion equation. The parameters encoded in a latent variable are fed into the PD along with a set of initial conditions, which the PD propagates forward in time based on the extracted physical parameter.

Here, we present a deep learning architecture shown in Figure 4 which is based on the DE-PD architecture. The DE takes in the full input series $\{\mathbf{x}_t\}_{t=0}^{T_x}$ and outputs a single-dimensional latent variable z . Unlike the original DE-PD architecture presented in [12], the DE here is not a VAE. The DE here consists of several convolutional layers followed by fully-connected layers and a batch normalization layer. More details are given in Appendix C. The parameter z along with an initial condition, $\mathbf{y}_0$, is fed into a PD which predicts the future time steps $\{\hat{\mathbf{y}}_t\}_{t=1}^{T_y}$. The PD consists of a “SEQL cell” in a recurrent structure, such that each step in the recurrent structure predicts a single time step forward. The SEQL cell consists of separate SEQL networks for each of feature in $\hat{\mathbf{y}}_t$.

The full architecture is trained end-to-end using a mean-squared error loss between the predicted dynamics $\{\hat{\mathbf{y}}_t\}_{t=1}^{T_y}$ and the target series $\{\mathbf{y}_t\}_{t=1}^{T_y}$. The datasets are derived from two different physical systems (kinematics and simple harmonic oscillator) as shown in the next sections.

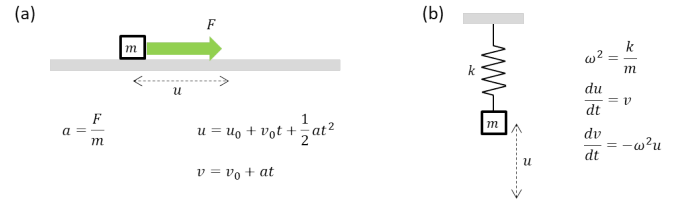


Fig. 5. (a) Kinematics describes the dynamics of an object where a force F is applied to a mass m . (b) Simple harmonic oscillator describes a mass m on a spring with spring constant k . In both cases, u is the displacement of the mass and v is the velocity.

1) *Kinematics*: Kinematics describes the motion of objects and is used in physics to study how objects move under some force. A schematic of a physical scenario described by kinematics is shown in Figure 5(a) in which an object on a frictionless surface has a force applied to it where the direction of the force is parallel to the surface. The relevant parameter to describe the object’s motion can be reduced to $a = \frac{F}{m}$ for a constant force F and object mass m . Given initial position and velocity u_0, v_0 , the object’s position u and velocity v at time t are given by

$$u(t) = u_0 + v_0 t + \frac{1}{2} a t^2$$

$$v(t) = v_0 + at$$

Acceleration a varies across different instances of the dataset. In our simulated dataset, we draw parameters from uniform distributions:

$$u_0, v_0, a \sim \mathcal{U}(-1, 1)$$

We set $\Delta t = 1$ such that u and v at each time step can be described in terms of the variables from the previous time step as

$$\begin{aligned} u_{i+1} &= u_i + v_i + \frac{1}{2}a \\ v_{i+1} &= v_i + a \end{aligned} \quad (1)$$

The initial parameters u_0, v_0 are fed into the propagator, and the dynamics encoder output is expected to correlate with a .

2) *Simple Harmonic Oscillator (SHO)*: The second physical system we analyze to demonstrate the dynamic system analysis architecture is the simple harmonic oscillator (SHO), a ubiquitous model in physics that can describe a wide range of physical systems including springs, pendulums, particles in a potential, or electric circuits. In general, the dynamics of the SHO can be given by the 2nd-order ordinary differential equation (ODE)

$$\frac{d^2 u}{dt^2} = -\omega^2 u \quad (2)$$

where ω is the resonant frequency of the system. In the case of a spring as shown in Figure 5(b), u is position and $\omega = \sqrt{k/m}$ where k is the spring constant and m is the mass of the object on the end of the spring. Equation 2 can be separated into two first-order ordinary differential equations:

$$\begin{aligned} \frac{du}{dt} &= v \\ \frac{dv}{dt} &= -\omega^2 u \end{aligned}$$

where v is the velocity.

The analytical solution of the state variables at each time step is

$$\begin{aligned} u_{i+1} &= u_i \cos(\omega \Delta t) + \frac{v_i}{\omega} \sin(\omega \Delta t) \\ v_{i+1} &= -u_i \omega \sin(\omega \Delta t) + v_i \cos(\omega \Delta t) \end{aligned}$$

where Δt is the time step and the subscripts i and $i+1$ mark the time at which the state variables are evaluated.

The SHO system can also be solved for numerically using a finite-difference approximation for the time derivatives. For example, the Euler method for integrating ODEs gives:

$$\begin{aligned} u_{i+1} &= u_i + v_i \Delta t \\ v_{i+1} &= v_i - \omega^2 u_i \Delta t \end{aligned} \quad (3)$$

In our experiments, we generate data with parameters drawn from uniform distributions:

$$\begin{aligned} u_0, v_0 &\sim \mathcal{U}(-1, 1) \\ \omega^2 &\sim \mathcal{U}(0.1, 1) \end{aligned}$$

The state variables are measured at a time step of $\Delta t = 0.1$ to allow the system to find the finite-difference solution. Because

of this small time step, we also need to propagate the solution for more time steps to find the right equation (otherwise the system learns the identity function). To avoid problems of the recurrent structure predicting a solution that blows up toward $\pm\infty$, we start the training with propagating only 1 time step, and add more time steps as the training continues. This is a similar strategy as [9] except that we are not restarting the training.

The initial parameters u_0, v_0 are fed into the propagator, and the dynamics encoder output is expected to correlate with ω^2 .

D. Training

The neural network is implemented in TensorFlow [23]. The network is trained using backpropagation with the RMSProp optimizer [24] and the following loss function:

$$\mathcal{L} = \frac{1}{N} \sum (y_i - \hat{y}_i)^2 + \lambda L_{0.5}^*$$

where N is the size of the training dataset and λ is a hyperparameter that balances the regularization versus the mean-squared error.

Similar to [6], we introduce a multi-phase training schedule. In an optional first phase, we train with a small value of λ , allowing for the parts of the network apart from the SEQL to evolve freely and extract the latent parameters during training. In the second phase, λ is increased to a point where it forces the SEQL network to become sparse. After this second phase, weights in the SEQL network below a certain threshold α are set to 0 and frozen such that they stay 0, equivalent to fixing the L_0 norm. In the final phase of training, the system continues training without $L_{0.5}^*$ regularization (i.e. $\lambda = 0$) and with a reduced maximum learning rate in order to fine-tune the weights.

Specific details for each experiment are listed in Appendix C.

IV. RESULTS

A. MNIST Arithmetic

Figure 6(b) plots the latent variable z versus the true label ψ for each digit after the entire network has been trained. Note that the system is trained on digits drawn from the MNIST training dataset and we also evaluate the trained network's performance on digits drawn from the MNIST test dataset. We see a strong linear correlation for both datasets, showing that the encoder has successfully learned a linear mapping to the digits despite not having access to the digit label ψ . Also note that there is a constant scaling factor between z and ψ due to the lack of constraint on z . A simple linear regression shows that the relation is

$$\psi = -1.788z + 4.519 \quad (4)$$

The extracted equation from the SEQL network for this result is shown in Table I. The ‘‘Encoder’’ equation is what we expect based on the encoder result in Equation 4. We conclude that the SEQL network has successfully extracted the additive nature of the function. Plotted in Figure 6(c-d)

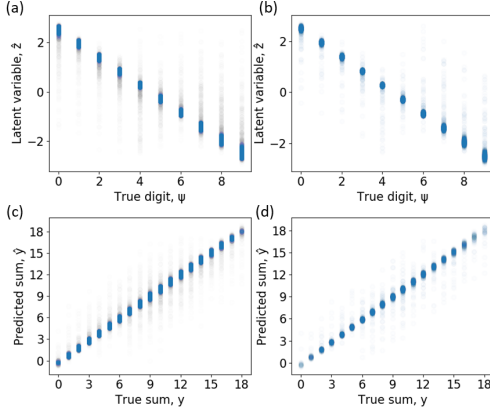


Fig. 6. The ability of the encoder to differentiate between digits as measured by the latent variable z versus the true digit ψ for digits χ drawn from the MNIST (a) training dataset and (b) test dataset. The correlation coefficients are -0.985 and -0.988 , respectively. The ability of the entire architecture to fit the label y as measured by the predicted sum $\hat{y}$ versus the true sum y for digits χ drawn from the MNIST (c) training dataset and (d) MNIST test dataset.

TABLE I
MNIST ARITHMETIC EXPECTED AND EXTRACTED EQUATIONS

True	$y = \psi_1 + \psi_2$
Encoder	$\hat{y} = -1.788z_1 - 1.788z_2 + 9.04$
SEQL	$\hat{y} = -1.809z_1 - 1.802z_2 + 9$

are the predicted sums $\hat{y}$ versus the true sums y . The mean absolute errors of prediction for the system drawing digits from the MNIST training and test datasets are 0.307 and 0.315, respectively.

While the architecture is trained as a regression problem using a mean square loss, we can still report accuracies as if it is a classification task since the labels y are integers. To calculate accuracy, we first round the predicted sum $\hat{y}$ to the nearest integer and then compare it to the label y . The trained system achieves accuracies of 89.7% and 90.2% for digits drawn from the MNIST training and test datasets, respectively.

To demonstrate the generalization of this architecture to data outside of the training dataset, we train the system using a scheme where MNIST digit pairs χ_1, χ_2 are randomly sampled from the MNIST training dataset and used as a training data point if they follow the condition $\psi_1 + \psi_2 < 15$. Otherwise, the pair is discarded. In the test phase, MNIST digit pairs χ_1, χ_2 are randomly sampled from the MNIST training dataset and kept in the evaluation dataset if $\psi_1 + \psi_2 \geq 15$. Otherwise, the pair is discarded.

For comparison, we also test the generalization of the encoder by following the above procedures but drawing MNIST digit pairs χ_1, χ_2 from the MNIST test dataset.

Generalization results of the network are shown in Table II. In this case, the SEQL network has learned the equation $\hat{y} = -1.56z_1 - 1.56z_2 + 8.66$. First, the most significant result is the difference between the accuracy evaluated on pairs $y < 15$ and pairs $y \geq 15$. For the architecture with the EQL network, the accuracy drops by a few percentage points. However, for

TABLE II
MNIST ARITHMETIC GENERALIZATION RESULTS

Source of w_i to form $x = \{w_1, w_2\}$	Network after the encoder	Accuracy [%]	
		$y < 15$	$y \geq 15$
MNIST training dataset	SEQL	92	87
	ReLU	93	0.8
MNIST test dataset	SEQL	91	83
	ReLU	92	0.6

the architecture where the SEQL network is replaced by the commonly used fully-connected network with ReLU activation functions (which we label as “ReLU”), the accuracy drops to below 1% showing that the results of the SEQL is able to generalize reasonably well in a regime where the ReLU cannot generalize at all. It is not necessarily an issue with the encoder since the system sees all digits 0 through 9.

Second, the accuracy drops slightly when digits are drawn from the MNIST test dataset versus when the digits are drawn from the MNIST training dataset, as expected. We did not optimize the hyperparameters of the digit extraction network since the drop in accuracy is small. Therefore, this could be optimized further if needed.

Finally, the accuracy drops slightly for pairs $y < 15$ when using the SEQL versus the ReLU network. This is unsurprising since the larger size and symmetric activation functions of the ReLU network constrains the network less than the SEQL and may make the optimization landscape smoother.

B. Kinematics

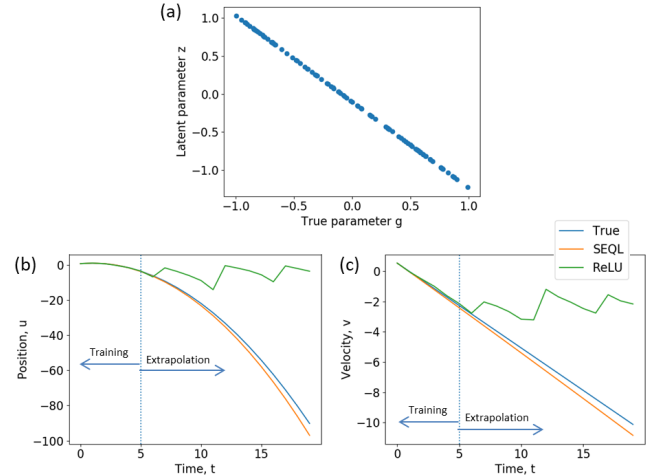


Fig. 7. (a) Latent parameter z of the dynamic encoder architecture after training plotted as a function of the true parameter a . We see a strong linear correlation. (b,c) Predicted propagation $\{\hat{y}_i\} = \{\hat{u}_i, \hat{v}_i\}$ with the SEQL cell and a ReLU network. “True” refers to the true propagation $\{y_i\}$.

Figure 7(a) shows the extracted latent parameter z plotted as a function of the true parameter a . We see a linear correlation with correlation coefficient close to -1 , showing that the

dynamics encoder has extracted the relevant parameter of the system. Again, there is a scaling relation between z and a :

$$a = -0.884z - 0.091 \quad (5)$$

An example of the equations found by the SEQL network after training is shown in Table III. The “DE” equation is what we expect based on latent variable extracted with the relation in Equation 5. These results match closely with what we expect.

TABLE III
KINEMATICS EXPECTED AND EXTRACTED EQUATIONS

True	$u_{i+1} = u_i + v_i + \frac{1}{2}a$ $v_{i+1} = v_i + a$
DE	$\hat{u}_{i+1} = u_i + v_i - 0.442z - 0.045$ $\hat{v}_{i+1} = v_i - 0.884z - 0.091$
SEQL	$\hat{u}_{i+1} = 1.002u_i + 1.002v_i - 0.475z$ $\hat{v}_{i+1} = 1.002v_i - 0.918z - 0.102$

The predicted propagation $\{\hat{y}_i\}$ is plotted in Figure 7(c-d). “True” is the true solution that we want to fit, and “SEQL” is the solution propagated by the SEQL network. For comparison, we also train a neural network with a similar architecture to the one shown in Figure 4 but where the SEQL cell is replaced by a standard fully-connected neural network with 2 hidden layers of 50 neurons each and ReLU activation functions (which we label as “ReLU”). While both networks match the true solution very closely in the training regime (left of the dotted line), the ReLU network quickly diverges from the true solution outside of the training regime. The SEQL cell is able to match the solution reasonably well for several more time steps, showing how it can extrapolate beyond the training data.

C. SHO

The plot of the latent variable z as a function of the true parameter ω^2 is shown in Figure 8(a). Note that there is a strong linear correlation between z and ω^2 rather than z and ω . This reflects the fact that using ω^2 requires fewer operations in the propagating equations than ω , the latter of which would require a squaring function. Additionally, the system was able to find that ω^2 is the simplest parameter to describe the system due to the sparsity regularization on the SEQL cell. We see a strong linear correlation with a correlation coefficient of -0.995 , showing that the dynamics encoder has successfully extracted the relevant parameter of the SHO system. A linear regression shows that the relation is:

$$\omega^2 = -0.927z + 0.464 \quad (6)$$

The equations extracted by the SEQL cell (consisting of 2 SEQL networks) are shown in Table IV. The “DE” equation is what we expect based on the dynamics encoder result in Equation 6. Immediately, we see that the expression for $\hat{u}_{i+1}$ and the first three terms of $\hat{v}_{i+1}$ match closely with the Euler method approximation using the latent variable relation extracted by the dynamics encoder.

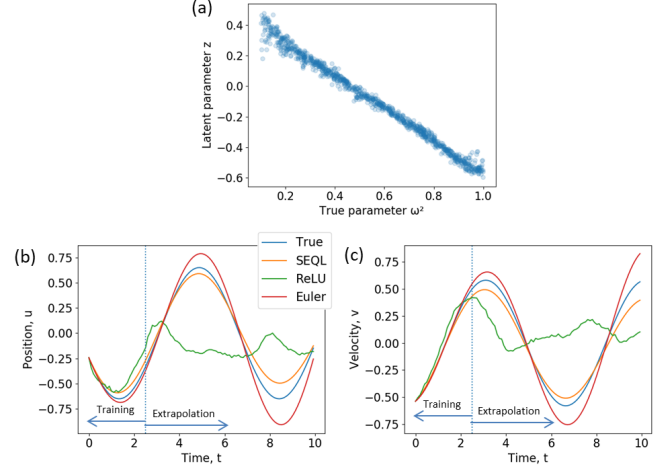


Fig. 8. (a) Simple harmonic oscillator model describing a mass m on a spring with spring constant k . The resonant frequency is related by $\omega^2 = \frac{k}{m}$. The state variables are vertical position of the mass u and velocity of the mass v . (b) Latent parameter z of the dynamic encoder architecture after training plotted as a function of the true parameter ω^2 . We see a good linear correlation. (c, d) Velocity v and position u as a function of time.

TABLE IV
SHO EXPECTED AND EXTRACTED EQUATIONS

True	$u_{i+1} = u_i + 0.1v_i$ $v_{i+1} = v_i - 0.1\omega^2 u_i$
DE	$\hat{u}_{i+1} = u_i + 0.1v_i$ $\hat{v}_{i+1} = v_i - 0.0464u_i + 0.0927u_i z$
DE, 2nd Order	$\hat{u}_{i+1} = u_i + 0.1v_i$ $\hat{v}_{i+1} = 0.998v_i - 0.0464u_i + 0.0927u_i z + 0.0046v_i z$
SEQL	$\hat{u}_{i+1} = 0.994u_i + 0.0992v_i - 0.0031$ $\hat{v}_{i+1} = 0.995v_i - 0.0492d + 0.084u_i z + 0.0037v_i z + 0.0133z^2$

An interesting point is that while we normally use the first-order approximation of the Euler method for integrating ODEs:

$$v_{i+1} = v_i + \Delta t \left. \frac{dv}{dt} \right|_{t=i} + \mathcal{O}(\Delta t^2)$$

it is possible to expand the approximation to find higher-order terms. If we expand the Euler method to its second-order approximation, we get:

$$\begin{aligned} v_{i+1} &= v_i + \Delta t \left. \frac{dv}{dt} \right|_{t=i} + \frac{1}{2} \Delta t^2 \left. \frac{d^2 v}{dt^2} \right|_{t=i} + \mathcal{O}(\Delta t^3) \\ &\approx v_i - \Delta t \omega^2 u_i - \frac{1}{2} \Delta t^2 \omega^2 v_i \end{aligned}$$

The expected equation based on the dynamics encoder result and assuming the 2nd order expansion is labeled as “DE, 2nd Order” in Table IV. It appears that the SEQL network in this case has not only found the first-order Euler finite-difference method, it has also added on another small term that corresponds to second-order term in the Taylor expansion of v_{i+1} . The last term found by the SEQL network, $0.0133z^2$

is likely from either cross-terms inside the network or a lack of convergence to exactly 0 and would likely disappear with another thresholding process.

The solution propagated through time is shown in Figure 8(b-c). As before, “ReLU” is the solution propagated by an architecture where the SEQL network is replaced by a standard neural network with 4 hidden layers of 50 units each and ReLU activation functions. For an additional comparison, we have also calculated the finite-difference solution using Euler’s method to integrate the true ODEs which is labeled as “Euler” in Figure 8(b).

We can see that within the training regime, all three methods fit the true solution reasonably well. However, the ReLU network completely fails to extrapolate beyond the training regime and essentially regresses to noise around 0. The Euler method and the SEQL network are both able to extrapolate reasonably well beyond the training regime, although they both start to diverge from the true solution due to the accumulated errors in the Euler method. Interestingly, the SEQL network solution has a smaller error than the finite-difference solution, demonstrating that the SEQL network was able to learn higher-order corrections to the first-order Euler method. This could possibly lead to discovery of more efficient integration schemes for differential equations that are difficult to solve through finite-difference.

V. CONCLUSION

We have shown how we can integrate symbolic regression with deep learning architectures and train the entire system end-to-end to take advantage of the powerful deep learning training techniques that have been developed. Namely, we show that we can learn arithmetic on MNIST digits where the system must learn to identify the images in an image recognition task while simultaneously extracting the mathematical expression that relates the digits to the answer. Additionally, we show that we can simultaneously extract an unknown parameter from a dynamical system while propagating a different system forward in time. In the SHO system, the results suggest that we can discover new techniques for integrating ODEs, potentially paving the way for improved integrators, such as integrators for stiff ODEs that may be difficult to solve with numerical methods.

One direction for future work is to make the system less sensitive to random initializations. As seen by the benchmark results of the SEQL network in Appendix A, the SEQL network is not always able to find the correct mathematical expression. This is because there are a number of local minima in the SEQL network that the network can get stuck in. Gradient-based optimization methods are only guaranteed to find local minima rather than global minima. This is not typically a concern for neural networks because the local minima are typically close enough in performance to the global minimum [25]. However, for the SEQL network, we want to find the true global minimum. In this work, we have alleviated this issue by increasing stochasticity through large learning rates and by decreasing the sensitivity to random initializations by duplicating activation functions. Additionally, we run multiple

trials and find the best results, either manually or through an automated system [6], [7]. In future work, it may be possible to find the true global minimum without resorting to multiple trials as it has been shown that over-parameterized neural networks with certain types of activation functions are able to reach the global minimum through gradient descent in linear time regardless of the random initialization [26].

Other directions for future work include expanding the capabilities of the system. For example, one could extend the system to support spatio-temporal systems so that we can discover underlying PDEs of a system or introduce parametric dependence in which unknown parameters have a time-dependence.

APPENDIX A SEQL NETWORK DETAILS

The activation functions in each hidden layer consist of:

$$[1(\times 2), g(\times 4), g^2(\times 4), \sin(2\pi g)(\times 2), \\ e^g(\times 2), \text{sigmoid}(20g)(\times 2), g_1 * g_2(\times 2)]$$

where the sigmoid function is defined as:

$$\text{sigmoid}(g) = \frac{1}{1 + e^{-g}}$$

and the $(\times i)$ indicated the number of times each activation function is duplicated. The sin and sigmoid functions have multipliers inside so that the functions more accurately represent their respective shapes inside the input domain of $x \in [-1, 1]$. Unless otherwise stated, these are the activation functions used for the other experiments as well.

We use two phases of training, where the first phase has a learning rate of 10^{-2} and regularization weight 5×10^{-3} for 2000 iterations. Small weights are frozen and set to 0 after the first phase. The second phase of training has a learning rate of 10^{-3} for 10000 iterations.

To benchmark our symbolic regression system, we choose a range of trial functions that our architecture can feasibly construct, train the network through 20 trials, and count how many times it reaches the correct answer. Benchmarking results are shown in Table V. As mentioned in section III-A, we only need the network to find the correct equation at least once since we can construct a system that automatically picks out the correct solution based on equation simplicity and test error.

APPENDIX B L_0 REGULARIZATION

We have also implemented an SEQL network that uses a relaxed form of L_0 regularization for neural networks introduced by [13]. We briefly review the details here, but refer the reader to [13] for more details.

The weights $\mathbf{W}$ of the neural network are reparameterized as

$$\mathbf{W} = \tilde{\mathbf{W}} \odot \mathbf{z}$$

where ideally each element of $\mathbf{z}$, $z_{j,k}$, is a binary “gate”, $z_{j,k} \in \{0, 1\}$. However, this is not differentiable and so we allow

TABLE V
BENCHMARK RESULTS FOR THE SEQL NETWORK.

Function	Success Rate	
	$L_{0.5}$	L_0
x	1	1
x^2	0.6	0.75
x^3	0.3	0.05
$\sin(2\pi x)$	0.45	0.85
xy	0.8	1
$\frac{1}{1+e^{-10x}}$	0.3	0.55
$\frac{xy}{2} + \frac{z}{2}$	0.05	0.95
$\exp(-x^2)$	0.05	0.15
$x^2 + \sin(2\pi y)$	0.2	0.8
$x^2 + y - 2z$	0.6	0.9

$z_{j,k}$ to be a stochastic variable drawn from the hard concrete distribution:

$$\begin{aligned}
 u &\sim \mathcal{U}(0,1) \\
 s &= \text{sigmoid}([\log u - \log(1-u) + \log \alpha_{j,k}]/\beta) \\
 \bar{s} &= s(\zeta - \gamma) + \gamma \\
 z_{j,k} &= \min(1, \max(0, \bar{s}))
 \end{aligned}$$

where $\alpha_{j,k}$ is a trainable variable that describes the location of the hard concrete distribution, and β, ζ, γ are hyperparameters that describe the distribution. In the case of binary gates, the regularization penalty would simply be the sum of $\mathbf{z}$ (i.e., the number of non-zero elements in $\mathbf{W}$). However, in the case of the hard concrete distribution, we can calculate an analytical form for the expectation of the regularization penalty over the distribution parameters. The total loss function is then

$$\mathcal{L} = \frac{1}{N} \sum (y_i - \hat{y}_i)^2 + \sum_{j,k} \text{sigmoid}\left(\log \alpha_{j,k} - \beta \log \frac{-\gamma}{\zeta}\right)$$

The advantage of L_0 regularization is that it enforces sparsity without placing a penalty on the magnitude of the weights. This also allows us to train the system without needing a final stage where small weights are set to 0 and frozen. While the reparameterization by [13] requires us to double the number of trainable parameters in the neural network, the regularization is only applied to the SEQL network which is small compared to the rest of the architecture.

In our experiments, we use the hyperparameters for the L_0 regularization suggested by [13], although these can be optimized in future work. Additionally, while [13] apply group sparsity to the rows of the weight matrices with the goal of computational efficiency, we apply parameter sparsity with the goal of simplifying the symbolic expression. We benchmark the SEQL network using L_0 regularization with the aforementioned trial functions and list the results in Table V. The success rates appear to be as good or better than the network using $L_{0.5}$ regularization for most of the trial functions that we have picked. We have also integrated the SEQL network using L_0 regularization into the MNIST arithmetic and kinematics architectures, and have found similar results as the SEQL network using $L_{0.5}$ regularization.

APPENDIX C EXPERIMENT DETAILS

A. MNIST Arithmetic

The encoder network consists of a convolutional layer with $32 \ 5 \times 5$ filters followed by a max pooling layer, another convolutional layer with $64 \ 5 \times 5$ filters followed by a max pooling layer, and 2 fully-connected layers with 128 and 16 units each with ReLU activation units. The max pooling layers have pool size of 2 and stride length of 2. The fully-connected layers are followed by 1-unit layer with batch normalization. The output of the batch normalization layer is divided by 2 such that the standard deviation of the output is 0.5. This decreases the range of the inputs to the SEQL network since the SEQL network was constructed assuming an input domain of $x \in [-1, 1]$. Additionally, the output of the SEQL network, $\hat{y}^*$, is scaled as $\hat{y} = 9\hat{y}^* + 9$ before being fed into the loss function so as to normalize the output against the range of expected y (this is equivalent to normalizing y to the range $[-1, 1]$).

The ReLU network that is trained in place of the SEQL network for comparison consists of two hidden layers with 50 units each and ReLU activation.

We use two phases of training, where the first phase uses a learning rate of 10^{-2} and regularization weight $\lambda = 0.05$. The second phase uses a learning rate of 10^{-4} and no regularization. The small weights are frozen between the first and second phase with a threshold of $\alpha = 0.01$. Each phase is trained for 10000 iterations.

B. Kinematics

To generate the kinematics dataset, we sample 100 values for a and generate a time series $\{\mathbf{x}_t\}_{t=0}^{T_x-1}$ and $\{\mathbf{y}_t\}_{t=0}^{T_y}$ for each a . The input series is propagated for $T_x = 100$ time steps.

The dynamics encoder consists of 2 1D convolutional layers with 16 filters of length 5 in each layer. These are followed by a hidden layer with 16 nodes and ReLU activation function, an output layer with one unit, and a batch normalization layer with standard deviation 0.5. The ReLU network that is trained in place of the SEQL network for comparison is the same as that of the MNIST task.

We use two phases of training, where the first phase uses a learning rate of 10^{-2} and a regularization weight of $\lambda = 10^{-3}$ for a total of 5000 iterations. The system is trained on $T_y = 1$ time step for the first 1000 iterations, and then $T_y = 5$ time steps for the remainder of the training. The small weights are frozen between the first and second phase with a threshold of $\alpha = 0.1$. The second phase uses a base learning rate of 10^{-3} and no regularization for 10000 iterations.

C. SHO

To generate the SHO dataset, we sample 1000 datapoints values for ω^2 and generate time series time series $\{\mathbf{x}_t\}_{t=0}^{T_x-1}$ and $\{\mathbf{y}_t\}_{t=0}^{T_y}$ for each ω^2 . The input series is propagated for $T_x = 500$ time steps with a time step of $\Delta t = 0.1$. The output

series is propagated for $T_y = 25$ time steps with the same time step.

The dynamics encoder is the same architecture as used in the kinematics experiment. Due to the greater number of time steps that the system needs to propagate, the SEQL network does not duplicate the activation functions for all functions. The functions in each hidden layer consist of:

$$[1(\times 2), g(\times 2), g^2, \sin(2\pi g), e^g, 10g_1 * g_2(\times 2)]$$

The ReLU network that is trained in place of the SEQL network for comparison consists of four hidden layers with 50 units each and ReLU activation functions.

We use three phases of training, where the first phase uses a learning rate of 10^{-2} and a regularization weight of $\lambda = 4 \times 10^{-5}$ for a total of 2000 iterations. The system starts training on $T_y = 1$ time steps for the first 500 time steps and then add 2 more time steps every 500 iterations for a total of $T_y = 7$ time steps. In the second phase of training, we increase the number of time steps to $T_y = 25$, decrease the base learning rate to 2×10^{-3} , and increase the regularization weight to $\lambda = 2 \times 10^{-4}$. The small weights are frozen between the second and third phase with a threshold of $\alpha = 0.01$. The third and final phase of training uses a base learning rate of 10^{-3} and no regularization.

ACKNOWLEDGMENT

We would like to acknowledge Joshua Tenenbaum, Max Tegmark, Jason Fleischer, Alexander Alemi, Jasper Snoek, Stepjan Picek, Rumen Dangovski, and Ilan Mitnikov for fruitful conversations. This research is sponsored in part by the Army Research Office and under Cooperative Agreement Number W911NF-18-2-0048, by the Department of Defense through the National Defense Science & Engineering Graduate Fellowship (NDSEG) Program, by the MITSenseTime Alliance on Artificial Intelligence, by the Defense Advanced Research Projects Agency (DARPA) under Agreement No. HR00111890042, and by MIT AIIA.

REFERENCES

- [1] J. Koza, "Genetic programming as a means for programming computers by natural selection," *Statistics and Computing*, vol. 4, no. 2, pp. 87–112, jun 1994. [Online]. Available: <http://link.springer.com/10.1007/BF00175355>
- [2] M. Schmidt and H. Lipson, "Distilling free-form natural laws from experimental data," *Science (New York, N.Y.)*, vol. 324, no. 5923, pp. 81–5, apr 2009. [Online]. Available: <http://www.ncbi.nlm.nih.gov/pubmed/19342586>
- [3] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proceedings of the National Academy of Sciences of the United States of America*, vol. 113, no. 15, pp. 3932–7, apr 2016. [Online]. Available: <http://www.ncbi.nlm.nih.gov/pubmed/27035946>
<http://www.pubmedcentral.nih.gov/articlerender.fcgi?artid=PMC4839439>
- [4] S. H. Rudy, S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Data-driven discovery of partial differential equations," *Science Advances*, vol. 3, no. 4, p. e1602614, apr 2017.
- [5] H. Schaeffer, "Learning partial differential equations via data discovery and sparse optimization," *Proceedings of the Royal Society A*, vol. 473, no. 2197, 2017. [Online]. Available: <http://rspa.royalsocietypublishing.org/content/royrspa/473/2197/20160446.full.pdf>
- [6] G. Martius and C. H. Lampert, "Extrapolation and learning equations," oct 2016. [Online]. Available: <http://arxiv.org/abs/1610.02995>
- [7] S. S. Sahoo, C. H. Lampert, and G. Martius, "Learning Equations for Extrapolation and Control," jun 2018. [Online]. Available: <https://arxiv.org/abs/1806.07259>
- [8] Z. Long, Y. Lu, X. Ma, and B. Dong, "PDE-Net: Learning PDEs from Data," in *Proceedings of Machine Learning Research*, jul 2018, pp. 3208–3216. [Online]. Available: <http://proceedings.mlr.press/v80/long18a.html>
- [9] Z. Long, Y. Lu, and B. Dong, "PDE-Net 2.0: Learning PDEs from Data with A Numeric-Symbolic Hybrid Deep Network," nov 2018. [Online]. Available: <http://arxiv.org/abs/1812.04426>
- [10] A. Trask, F. Hill, S. Reed, J. Rae, C. Dyer, and P. Blunsom, "Neural Arithmetic Logic Units," aug 2018.
- [11] D. Zheng, V. Luo, J. Wu, and J. B. Tenenbaum, "Unsupervised learning of latent physical properties using perception-prediction networks," in *34th Conference on Uncertainty in Artificial Intelligence 2018, UAI 2018*, vol. 1. Association For Uncertainty in Artificial Intelligence (AUAI), 2018, pp. 497–507.
- [12] P. Y. Lu, S. Kim, and M. Soljačić, "Extracting Interpretable Physical Parameters from Spatiotemporal Systems using Unsupervised Learning," jul 2019. [Online]. Available: <http://arxiv.org/abs/1907.06011>
- [13] C. Louizos, M. Welling, and D. P. Kingma, "Learning Sparse Neural Networks through $\mathcal{L}_0$ Regularization," dec 2017. [Online]. Available: <https://arxiv.org/abs/1712.01312>
- [14] D. Molchanov, A. Ashukha, and D. Vetrov, "Variational dropout sparsifies deep neural networks," in *34th International Conference on Machine Learning, ICML 2017*, vol. 5. International Machine Learning Society (IMLS), 2017, pp. 3854–3863.
- [15] M. Zhu and S. Gupta, "To prune, or not to prune: exploring the efficacy of pruning for model compression," oct 2017. [Online]. Available: <http://arxiv.org/abs/1710.01878>
- [16] T. Gale, E. Elsen, and S. Hooker, "The State of Sparsity in Deep Neural Networks," feb 2019. [Online]. Available: <http://arxiv.org/abs/1902.09574>
- [17] B. K. Natarajan, "Sparse Approximate Solutions to Linear Systems," *SIAM Journal on Computing*, vol. 24, no. 2, pp. 227–234, apr 1995. [Online]. Available: <http://epubs.siam.org/doi/10.1137/S0097539792240406>
- [18] S. Srinivas, A. Subramanya, and R. V. Babu, "Training Sparse Neural Networks," in *2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE, jul 2017, pp. 455–462. [Online]. Available: <http://ieeexplore.ieee.org/document/8014795/>
- [19] Z. Xu, H. Zhang, Y. Wang, X. Chang, and Y. Liang, "L 1/2 regularization," *Science China Information Sciences*, vol. 53, no. 6, pp. 1159–1169, jun 2010. [Online]. Available: <http://link.springer.com/10.1007/s11432-010-0090-0>
- [20] Z.-B. Xu, H.-L. Guo, Y. Wang, and H. Zhang, "Representative of L1/2 Regularization among L_q ($0 < q \leq 1$) Regularizations: an Experimental Study Based on Phase Diagram," *Acta Automatica Sinica*, vol. 38, no. 7, pp. 1225–1228, jul 2012. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1874102911602930>
- [21] Q. Fan, J. M. Zurada, and W. Wu, "Convergence of online gradient method for feedforward neural networks with smoothing L1/2 regularization penalty," *Neurocomputing*, vol. 131, pp. 208–216, may 2014. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231213010825>
- [22] W. Wu, Q. Fan, J. M. Zurada, J. Wang, D. Yang, and Y. Liu, "Batch gradient method with smoothing L1/2 regularization for training of feedforward neural networks," *Neural Networks*, vol. 50, pp. 72–78, feb 2014. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0893608013002700>
- [23] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org. [Online]. Available: <http://tensorflow.org/>
- [24] T. Tieleman and G. Hinton, "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude," *COURSERA: Neural networks for machine learning*, vol. 4, no. 26–31, 2012.
- [25] A. Choromanska, M. Henaff, M. Mathieu, G. B. Arous, and Y. LeCun, "The loss surfaces of multilayer networks," in *Journal of Machine Learning Research*, vol. 38. Microtome Publishing, 2015, pp. 192–204.

- [26] S. S. Du, J. D. Lee, H. Li, L. Wang, and X. Zhai, “Gradient Descent Finds Global Minima of Deep Neural Networks,” nov 2018. [Online]. Available: <http://arxiv.org/abs/1811.03804>